A Regression-Guided Superoptimizer for a Reduced RISC-V ISA

Project Draft — Artificial Intelligence Course Project

Abstract

This project proposes the design and implementation of a superoptimizer for a reduced, integer-only subset of the RISC-V (RV32I) instruction set architecture, excluding floating-point and integer division/remainder instructions. Rather than relying solely on brute-force enumeration or a hand-tuned cost model, the system uses a regression model, trained on labeled instruction-sequence data, to predict execution cost and guide the search for shorter or faster equivalent code sequences. This document lays out the motivation, scope, background concepts, a survey of related work, the proposed system architecture, and an evaluation plan.

1. Motivation and Problem Statement

Traditional compiler optimizers rely on a fixed catalogue of rewrite rules (peephole optimizations, strength reduction, constant folding) applied in a pass pipeline. These rules are correct by construction but incomplete: they cannot discover an optimization the compiler author never anticipated. A **superoptimizer** instead treats optimization as a search problem — given a target instruction sequence, search the space of all semantically equivalent sequences for one that is faster, shorter, or otherwise cheaper under some cost metric. Superoptimizers have been shown to discover non-obvious, algorithmically distinct code that outperforms production compilers on short basic blocks.

The main obstacle to practical superoptimization is search cost: evaluating the true performance of a candidate sequence (via cycle-accurate simulation or hardware execution) is slow, and the search may need to evaluate thousands to millions of candidates. This project's core idea is to replace or supplement the expensive evaluation step with a learned **regression model** that predicts a candidate sequence's cost directly from its features, making the search loop dramatically cheaper while remaining a reasonably accurate proxy for real performance.

2. Scope

2.1 Target ISA

The project targets a deliberately reduced subset of RV32I (32-bit RISC-V integer base ISA):

- **Included:** arithmetic (ADD, SUB, ADDI), logical (AND, OR, XOR, ANDI, ORI, XORI), shifts (SLL, SRL, SRA and immediate forms), comparisons (SLT, SLTU, SLTI), load/store (LW, LB, SW, SB), branches (BEQ, BNE, BLT, BGE), and unconditional jumps (JAL, JALR).

- **Excluded:** all floating-point instructions (RV32F/D extensions) and integer division/remainder (DIV, DIVU, REM, REMU from the M extension).
- **Rationale for exclusion:** integer arithmetic without division has exact, total semantics — no rounding modes, no NaNs, no division-by-zero edge cases — which makes equivalence checking by random testing far more reliable and keeps the instruction/operand space small enough to enumerate exhaustively for short sequences. This is what makes the project tractable within a single course term.

2.2 What the project will deliver

- A candidate-generation and search module (enumerative and/or mutation-based) over the restricted ISA.
- An equivalence checker that verifies a candidate sequence is behaviorally identical to the original (via exhaustive or randomized testing over the small operand space, optionally cross-checked with an SMT solver such as Z3 for short sequences).
- A regression-based cost model, trained on (instruction-sequence-features $\rightarrow$ measured cost) pairs, used to rank and prune candidates during search instead of invoking a cycle-accurate simulator every time.
- An evaluation comparing superoptimized output against GCC/Clang -O0 and -O2 output on a benchmark set of short basic blocks.

2.3 Explicit non-goals

- No support for loops, function calls, or whole-program optimization — the search is restricted to short, loop-free basic blocks (the same scope as STOKE's original loop-free formulation).
- No floating-point or division support, as noted above.
- No claim of formal completeness — the stochastic/enumerative search is a best-effort heuristic search, not a proof of optimality.

3. Background Concepts

3.1 Superoptimization

Superoptimization was introduced by Massalin (1987), who used brute-force enumeration of increasingly long instruction sequences, checking each for equivalence against a specification. Because the search space grows combinatorially with sequence length and instruction-set size, later work introduced smarter search strategies: **enumerative synthesis** (systematically explore short sequences, often pruned using semantic hashing), **stochastic search** (random mutation guided by a cost function, typically via Markov Chain Monte Carlo), and **symbolic/SMT-guided search** (use a solver to construct or verify candidates directly).

3.2 Cost models: simulated vs. learned

Every superoptimizer needs a way to score a candidate sequence's quality. Simulation-based cost models (e.g. LLVM-MCA, or a cycle-accurate simulator like gem5/Spike) are accurate but slow to query,

since each call requires modeling pipeline behavior. Learned (regression) cost models instead extract a feature vector from a candidate sequence — instruction counts, opcode histograms, data-dependency-chain depth, register pressure, memory-operation counts — and predict a scalar cost using a model trained on prior simulated or measured examples. Once trained, a learned model is orders of magnitude faster to query than a simulator, which is what makes it practical to embed inside a tight search loop that evaluates many candidates per second.

3.3 Equivalence checking

A rewritten sequence must be verified to compute the same result as the original for every possible input. For a small, total (no undefined behavior) instruction set like the one proposed here, this can be done cheaply by exhaustive or randomized testing across the operand space; an SMT solver (e.g. Z3) can additionally be used to prove equivalence exactly for short sequences, giving a correctness guarantee beyond what random testing alone provides.

3.4 Search strategies at a glance

Strategy	Idea	Trade-off
Enumerative	Exhaustively try all sequences up to length N	Complete but explodes combinatorially
Stochastic (MCMC)	Randomly mutate + accept/reject via cost function	Scales better, no completeness guarantee
Symbolic/SMT-guided	Solver constructs or verifies candidates	Precise, but solver calls are expensive
Learning-guided	ML model prioritizes which mutations/candidates to try	Needs training data, but focuses search

4. Related Work

The literature spans four generations of superoptimization research: foundational brute-force methods, stochastic search, retargetable frameworks, and — most recently — learning-guided and LLM-based approaches. Representative papers, ordered roughly chronologically:

4.1 Foundational and classical approaches

[1] H. Massalin, "Superoptimizer: A Look at the Smallest Program," ASPLOS 1987. The original superoptimizer: brute-force enumeration of short instruction sequences, checked for equivalence. Establishes the core problem formulation this project builds on.

[2] S. Bansal and A. Aiken, "Automatic Generation of Peephole Superoptimizers," ASPLOS 2006. Introduces an enumerative approach that builds a database of optimal short sequences, usable as a peephole pass — directly relevant to a restricted-ISA, basic-block-scale project like this one.

4.2 Stochastic search

[3] E. Schkufza, R. Sharma, and A. Aiken, "Stochastic Superoptimization," ASPLOS 2013 (arXiv:1211.0557). Introduces STOKE: formulates loop-free binary superoptimization as a stochastic search problem, using a cost function balancing correctness and performance, sampled with Markov Chain Monte Carlo. STOKE-produced code matched or beat GCC/ICC -O3 and, on some benchmarks, expert hand-written assembly. This is the most direct methodological template for the proposed project's search loop.

4.3 Retargetable / multi-ISA frameworks

[4] P. M. Phothilimthana, A. Thakur, R. Bodik, and D. Dhurjati, "GreenThumb: Superoptimizer Construction Framework," UC Berkeley Technical Report UCB/EECS-2016-8, 2016. A framework combining enumerative, stochastic, and SMT-based search, retargetable to a new ISA by implementing only an emulator for it — demonstrated on ARM, GreenArrays, and a subset of LLVM IR. Useful as an architectural reference for how to structure an ISA-agnostic search engine that could, in principle, be pointed at the reduced RISC-V subset proposed here.

4.4 Learned cost models for search-based optimization

[5] COMET: A Neural Cost Model Explanation Framework, MLSys 2024 (arXiv:2302.06836). Surveys and analyzes learned cost models used to guide compiler optimization and superoptimization, contrasting them with traditional simulation-based cost models such as LLVM-MCA and uiCA — directly relevant background for justifying a regression-based cost model.

[6] Z. Chen et al., "Ansor: Generating High-Performance Tensor Programs for Deep Learning," OSDI 2020 (arXiv:2006.06762). Uses a learned cost model to score candidate program schedules during search instead of measuring them directly, trained on extracted arithmetic and memory-access features. This project's planned regression model over instruction-sequence features follows the same pattern, applied at the instruction level instead of the tensor-schedule level.

4.5 Learning-guided and LLM-based superoptimization (2023–2025)

[7] Learning Guided Enumerative Synthesis for Superoptimization (with follow-on citing work through 2025, e.g. "Neural-Guided Superoptimization in Ethereum," Information and Software Technology, 2025). Uses a learned model to guide which candidates an enumerative synthesizer explores first, rather than exploring the space blindly — closely related to how this project's regression model is meant to prioritize search.

[8] SuperCoder: Assembly Program Superoptimization with Large Language Models, 2025 (arXiv:2505.11480). Trains an LLM with reinforcement learning (PPO/GRPO) using a reward function combining correctness and measured speedup, applied directly to assembly-level superoptimization. Representative of the current frontier (RL/LLM-driven superoptimization) and useful in a literature review to contrast with this project's simpler, more tractable regression-based approach.

[9] Pearl: Automatic Code Optimization using Deep Reinforcement Learning, ACM ICS 2025. Applies deep RL to automatic code optimization, illustrating an alternative to supervised regression for the cost/guidance model, worth mentioning as a stretch-goal extension.

4.6 RISC-V and machine-learning-on-hardware context

[10] "RISC-V and Machine Learning: A Survey," The Journal of Supercomputing, 2026. A broad survey of ML on RISC-V (accelerator design, framework support, benchmarking), useful for framing why RISC-V is a relevant and active ISA to target, and for pointing to open tooling (simulators, compilers) available for the implementation phase.

Positioning of this project: most of the papers above either (a) target a full, complex ISA (x86-64, ARM) with expensive verification, or (b) use heavyweight learned models (transformers, RL over LLMs) that are out of scope for a course project. This project's contribution is deliberately narrow: combine a classical search strategy (enumerative and/or MCMC, following Massalin/Bansal-Aiken/STOKE) with a lightweight regression cost model (following the Ansor/COMET line of work) on an ISA subset small enough that correctness can be checked exhaustively and the whole pipeline can be built and evaluated in one term.

5. Proposed Methodology

5.1 System architecture

The system consists of four components, run in a loop:

- **Candidate generator** — proposes a new instruction sequence, either by exhaustive enumeration (short sequences) or by mutating the current best candidate (instruction swap, operand change, insertion/deletion) for longer sequences.
- **Equivalence checker** — verifies the candidate computes the same result as the original over the relevant input space (exhaustive/random testing; optionally Z3 for a formal check on short sequences). Candidates that fail are discarded immediately.
- **Regression cost model** — takes a feature vector extracted from the candidate (instruction count, opcode histogram, dependency-chain depth, memory-op count, branch count, register reuse) and predicts a scalar cost. Trained offline on a labeled dataset of (features, measured cost) pairs.
- **Search controller** — accepts or rejects candidates based on predicted cost (e.g. simulated-annealing acceptance criterion for MCMC, or best-first ranking for enumerative search), keeping the best verified candidate found so far.

5.2 Building the regression training set

- Generate a large set of random or systematically varied short instruction sequences within the ISA subset.
- Measure their true cost via a RISC-V simulator (e.g. Spike, or a simple in-house cycle counter) — instruction count and estimated cycle count are both reasonable starting labels.
- Extract the feature vector described above for each sequence.
- Train and compare several regression models, in increasing complexity — start with linear regression as a baseline, then a tree-based ensemble (e.g. gradient-boosted trees), to demonstrate a proper AI-course progression and quantify the accuracy/speed trade-off between models.

5.3 Evaluation plan

- **Correctness:** 100% of accepted candidates must pass the equivalence checker.
- **Cost-model accuracy:** report R-squared / mean absolute error of predicted vs. simulated cost on a held-out test set of sequences.
- **End-to-end quality:** compare superoptimized output against GCC/Clang -O0 and -O2 on a benchmark set of short basic blocks (arithmetic kernels, bit-manipulation idioms), measuring instruction-count and estimated-cycle reduction.
- **Search efficiency:** compare wall-clock search time using the regression cost model versus using the simulator directly inside the loop, to quantify the speedup the learned model provides.

6. Suggested Timeline

Phase	Task
Weeks 1–2	ISA subset definition, instruction semantics, emulator/interpreter for the subset
Weeks 3–4	Equivalence checker (random testing + optional Z3), basic enumerative search over short sequences
Weeks 5–6	Generate training data (simulated costs), feature extraction, train baseline regression model
Weeks 7–8	Integrate regression model into search loop; add mutation-based/stochastic search for longer sequences
Weeks 9–10	Evaluation against GCC/Clang output; ablations (model complexity, search strategy)
Weeks 11–12	Write-up, polish, prepare presentation/demo

7. Risks and Open Questions

- **Training data cost:** generating enough labeled (sequence, cost) pairs may require a working simulator early — budget time for this in weeks 1–2.
- **Cost-model fidelity:** a regression model may miss microarchitectural effects (pipelining, out-of-order execution); mitigate by using instruction count / a simple in-order cycle model as the label rather than a full out-of-order simulator, which keeps the label itself easier to learn.
- **Search space size:** even a reduced ISA has a large combinatorial space for sequences longer than ~6 instructions — enumerative search should be capped in length, with mutation-based search taking over beyond that.
- **Scope creep:** loops and function calls are explicitly out of scope; keep the benchmark set to loop-free basic blocks to stay within a one-term timeline.

Draft for review — feedback welcome on scope, timeline, and choice of regression model.